

Leader Election under Crash Recovery Fault Models

Ishank Sharma, Srilalitha Nanjangud Subbaswamy, Prerana Puttaswamy

Department of Computer Science, California State University, Long Beach

Long Beach, CA, USA

Email: {ishank.sharma01, srilalitha.nanjangud, prerana.puttaswamy}@student.csulb.edu

Abstract—Leader election is a fundamental building block of distributed systems, yet many classical evaluations assume failure-free execution or simplified crash models. In this project we adopt the role of *researchers* and perform a survey-driven empirical study of leader election under an explicit crash-recovery and network-fault model. We (i) re-implement three canonical election strategies—Bully, Ring, and the election phase of Raft—in a common discrete-event simulator, and (ii) extend this baseline with a new *Multi-Attribute* election algorithm that selects leaders based on node capability scores (battery and CPU). The simulator models a 10-node cluster with 50 ms one-way latency, ± 10 ms jitter, 5% independent packet loss, and a fault script consisting of one leader crash, a 1.5 s network partition, and a leader restart. Across five random seeds we observe that Bully, Ring, Raft, and Multi-Attribute achieve median (P50) re-election latencies of 0.81 s, 3.01 s, 2.29 s, and 0.73 s respectively, with average election-related message counts of 1168, 463, 284, and 573. Ring succeeds in only 1/5 runs under partitions, whereas the other three algorithms converge in 5/5 runs. The Multi-Attribute algorithm reduces re-election latency by 10% relative to Bully while using approximately 51% fewer messages, and matches Raft’s robustness. We present the simulator architecture, discuss how the observed behaviors arise from the interaction of timeouts, partitions, and leader stability, and outline limitations and avenues for further experimental extensions.

Index Terms—leader election, crash-recovery, distributed systems, Raft, Bully algorithm, simulation, network partitions, multi-attribute scoring

I. INTRODUCTION

Leader-based coordination is widely used in replicated databases, metadata stores, lock managers, and control planes. A leader is responsible for sequencing operations and orchestrating recovery. When the leader fails or becomes unreachable, the system must elect a new leader quickly and safely; otherwise clients observe downtime, inconsistent views, or conflicting writes.

In undergraduate textbooks, leader election is often introduced using relatively clean models: crash failures without restarts, reliable communication, and the absence of network partitions. In contrast, real deployments see crash-recovery behavior, packet loss, and temporary partitions. Understanding *how specific algorithms behave under such conditions* is therefore an important step towards robust design.

The goal of this project is to go beyond a programming assignment and carry out a small but coherent research-style study. Concretely, we:

- **Survey** recent work on multi-attribute, hierarchical, adaptive-timeout, and knowledge-free leader election [1]–[4].

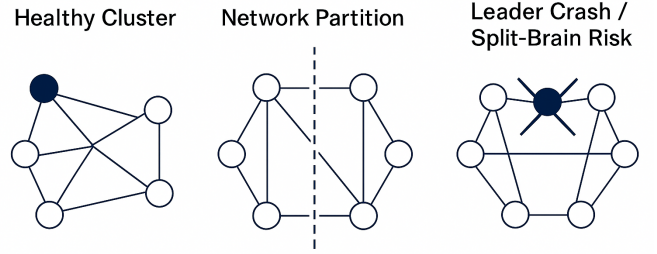


Fig. 1. Three key scenarios explored in the simulator: (left) healthy cluster with one leader; (middle) network partition; (right) leader crash and risk of split-brain.

- **Repeat** classical Bully, Ring, and Raft-style elections in a common simulator under a fixed crash-recovery fault script.
- **Extend** the space of algorithms with a *Multi-Attribute* variant that uses dynamic capability scores to choose leaders.

Our central research question is:

Given a 10-node cluster with crash-recovery, network partitions, jitter, and packet loss, which election algorithm yields the best combination of (i) low re-election latency, (ii) limited message overhead, and (iii) robustness against split-brain?

Fig. 1 provides an overview of the three main failure scenarios considered in the simulator.

Our results can be summarized as follows. Under the fault script described in Section III, Bully and Multi-Attribute re-elect leaders in under 1 s on median, but Bully sends roughly twice as many messages. Raft uses the fewest messages but takes longer to recover because it waits for majority quorums. Ring is fragile under partitions: it succeeds in only 20% of runs and exhibits median re-election latency above 3 s. These findings align qualitatively with the intuitions in the surveyed literature while providing concrete quantitative evidence.

II. RELATED WORK AND PROBLEM SETTING

We briefly position our work relative to prior research, using only the papers discussed in our project slides.

Multi-attribute self-stabilizing leader election schemes [1] assign each process a multi-dimensional capability vector (e.g., CPU, battery, connectivity) and define a ranking function over these vectors. From any initial configuration, the system converges to a state where the process with maximal rank becomes leader. Our Multi-Attribute algorithm is a simplified, non-self-stabilizing version oriented towards crash–recovery evaluation.

Hierarchical adaptive election mechanisms [2] reduce election traffic in large clusters by organizing nodes into subgroups that elect local leaders, followed by a second stage among those leaders. While our simulator uses only $n = 10$ nodes and does not implement hierarchy, we borrow the idea that election overhead must be considered alongside latency.

Dynamic timeout tuning [3] studies how to adapt election timeouts to measured network conditions. Long timeouts hurt recovery time; short timeouts increase false positives and unnecessary elections. Our experiments use fixed timeouts calibrated to our latency model but we discuss their impact in Section VIII.

Knowledge-free resilient election algorithms [4] seek correctness without assuming knowledge of the system size or topology, which is valuable in peer-to-peer or mobile ad-hoc networks. In contrast, our algorithms assume a fixed, known set of nodes, but our simulator could be extended to explore knowledge-free variants.

From a problem-setting perspective, we focus on a small but non-trivial cluster size ($n = 10$), explicit crash–recovery behavior, and adversarial yet deterministic fault timelines. This keeps the experiments tractable while still exposing interesting interactions between timeouts, partitions, and leader stability.

III. SYSTEM AND FAULT MODEL

We consider a set $N = \{0, 1, \dots, 9\}$ of ten nodes connected by a logical network. Nodes exchange messages over reliable FIFO channels subject to delay and loss.

A. Network Parameters

The network model has the following parameters, chosen to approximate a moderately fast but imperfect local cluster:

- **Base latency:** 50 ms one-way propagation delay.
- **Jitter:** an independent uniform offset in $[-10, +10]$ ms added to each message.
- **Loss:** each message is independently dropped with probability 0.05.

These values are injected at the message bus layer, described in Section IV.

B. Crash–Recovery and Partitions

A node may crash and later recover. A crash removes the node from the scheduler; it stops sending and processing messages. A recovery reinitializes its volatile state but retains static configuration such as its identifier and capabilities.

We use a fixed fault script applied uniformly to all algorithms:

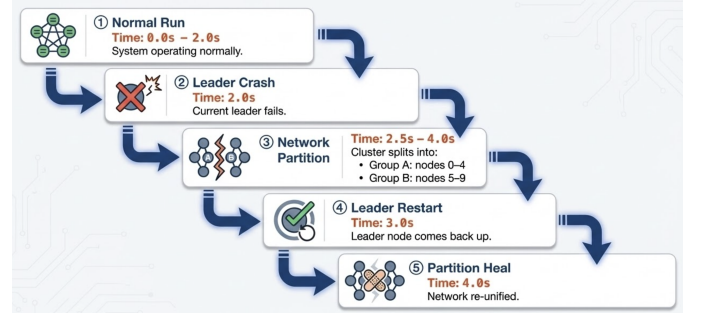


Fig. 2. Fault timeline used for all experiments: a single crash, partition, leader restart, and heal.

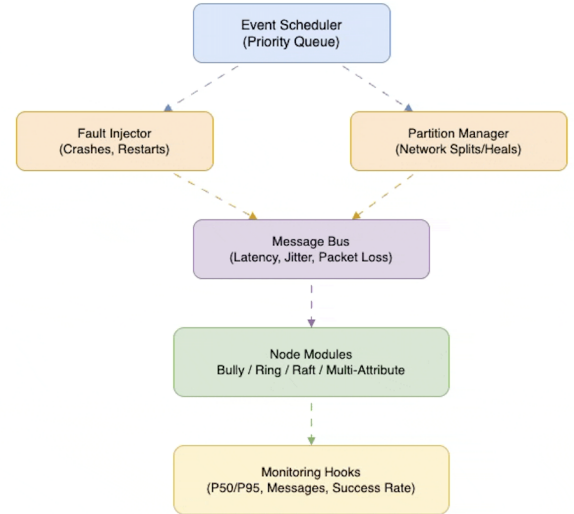


Fig. 3. Simulator architecture: node modules implement election algorithms; an event scheduler drives message delivery and timeouts; a fault injector and partition manager modify network connectivity; a monitor collects metrics.

- $t = 0.0$ s: cluster starts with no leader; each algorithm initiates an initial election.
- $t = 2.0$ s: the current leader crashes.
- $t = 2.5$ s: the network is partitioned into two disjoint sets, $N_1 = \{0, \dots, 4\}$ and $N_2 = \{5, \dots, 9\}$, between which all messages are dropped.
- $t = 3.0$ s: the crashed leader restarts inside its original partition.
- $t = 4.0$ s: the partition heals and full connectivity is restored.

This sequence stresses both failure detection and reconciliation after conflicting election attempts.

IV. SIMULATOR ARCHITECTURE

Our simulator is a single-process, discrete-event engine that executes a sequence of timestamped events. Fig. 3 shows the top-level architecture.

A. Node Module

Each node is modeled as an object exposing the following interface:

- `on_message(msg)` — handle incoming messages.
- `on_timeout(type)` — handle timer events (e.g., heartbeat, election).
- `on_crash()` and `on_restart()` — perform local state transitions on crash and recovery.

The concrete behavior of these callbacks depends on the election algorithm (Bully, Ring, Raft, or Multi-Attribute). The node module is oblivious to the global topology and fault script; it only sees messages and timers.

B. Event Scheduler

The scheduler maintains a min-heap of pending events keyed by simulated time. Each event contains:

- a timestamp,
- a target node or subsystem,
- an event type (message arrival, timeout, crash, restart, partition-change),
- a payload (for message arrivals).

Simulation proceeds by repeatedly popping the earliest event, updating the global clock, and invoking the corresponding handler.

C. Message Bus and Partition Manager

When a node sends a message to a peer, the message bus samples a delay and loss outcome according to the network parameters. If the message is not dropped, it schedules a delivery event at time $t_{\text{now}} + \Delta$. The partition manager stores a mapping from each node to its current partition and informs the message bus whether communication between a given source and destination is allowed. During a partition, any cross-partition message is dropped deterministically.

D. Fault Injector and Monitor

The fault injector enqueues crash, restart, and partition-change events according to the timeline in Section III. The monitor records the following metrics:

- time when the first leader is elected;
- time when a new leader is elected after the crash at $t = 2.0$ s;
- total number of election-related messages;
- whether a unique leader exists at the end of the run.

The simulator is parameterized by a random seed that influences message delays and loss events, allowing us to compute distributional statistics across runs.

V. ELECTION ALGORITHMS

We model four algorithms: Bully, Ring, Raft-style election, and a new Multi-Attribute variant. Bully and Ring follow standard textbook logic; Raft includes only the election phase (no log replication). Here we focus on the Multi-Attribute algorithm, which is the main extension introduced by this project.

Algorithm 1 Multi-Attribute Leader Election (Node i)

Constants: N (set of all nodes), T_{elect} , $T_{\text{heartbeat}}$

Local state: id_i , $score_i$, $state_i \in \{\text{FOLLOWER, CANDIDATE, LEADER}\}$, $leader_i$

- 1: **procedure** INITIALIZE
- 2: $battery \leftarrow \text{READBATTERYLEVEL}$
- 3: $cpu \leftarrow \text{READCPULOAD}$
- 4: $score_i \leftarrow 0.7 \cdot battery + 0.3 \cdot (100 - cpu)$
- 5: $state_i \leftarrow \text{FOLLOWER}; leader_i \leftarrow \perp$
- 6: $\text{STARTTIMER}(T_{\text{heartbeat}})$
- 7: **procedure** ONHEARTBEATTIMEOUT
- 8: $state_i \leftarrow \text{CANDIDATE}$
- 9: BROADCAST(ELECTION, $score_i$, id_i to all $j \in N, j \neq i$)
- 10: $\text{STARTTIMER}(T_{\text{elect}})$
- 11: **procedure** ONRECEIVE(m)
- 12: **if** $m.type = \text{ELECTION}$ **then**
- 13: **if** $score_i > m.score$ **or** ($score_i = m.score \wedge id_i > m.sender_id$) **then**
- 14: SEND(OK to $m.sender_id$)
- 15: **if** $state_i \notin \{\text{CANDIDATE, LEADER}\}$ **then**
- 16: ONHEARTBEATTIMEOUT
- 17: **else if** $m.type = \text{OK}$ **then**
- 18: $state_i \leftarrow \text{FOLLOWER}$
- 19: CANCELTIMER(T_{elect})
- 20: $\text{STARTTIMER}(T_{\text{heartbeat}})$
- 21: **else if** $m.type = \text{COORDINATOR}$ **then**
- 22: $leader_i \leftarrow m.sender_id$
- 23: $state_i \leftarrow \text{FOLLOWER}$
- 24: CANCELTIMER(T_{elect})
- 25: $\text{STARTTIMER}(T_{\text{heartbeat}})$
- 26: **else if** $m.type = \text{HEARTBEAT}$ **then**
- 27: $leader_i \leftarrow m.sender_id$
- 28: RESETTIMER($T_{\text{heartbeat}}$)
- 29: **procedure** ONELECTIONTIMEOUT
- 30: **if** $state_i = \text{CANDIDATE}$ **then**
- 31: $state_i \leftarrow \text{LEADER}; leader_i \leftarrow id_i$
- 32: BROADCAST(COORDINATOR, id_i to all $j \in N$)
- 33: STARTHEARTBEATLOOP

A. Multi-Attribute Leader Election

Each node i maintains a capability score $score_i$ derived from its battery level and CPU load. We use a linear combination

$$score_i = 0.7 \cdot battery_i + 0.3 \cdot (100 - cpu_i), \quad (1)$$

where battery and CPU are percentages in $[0, 100]$. The parameters 0.7 and 0.3 were chosen empirically to emphasize energy availability.

Nodes track their local state (FOLLOWER, CANDIDATE, or LEADER), a current leader identifier, and two timers: $T_{\text{heartbeat}}$ for detecting leader failure, and T_{elect} for bounding the election phase.

Algorithm 1 presents the pseudocode used in the simulator in algorithmic form.

TABLE I
MEDIAN ELECTION METRICS ACROSS FIVE SEEDS

Algorithm	L_{init} (P50, s)	L_{re} (P50, s)	M (avg.)	S (/5)
Bully	0.05	0.81	1168	5
Ring	2.00	3.01	463	1
Raft	0.11	2.29	284	5
Multi-Attribute	0.35	0.73	573	5

Compared to standard Bully:

- ELECTION messages are broadcast to *all* nodes because capability scores are not ordered by identifiers.
- Preemption is based on $score_i$ rather than id_i , with id_i used only as a tie-breaker.
- The resulting leader is usually among the highest-scoring nodes, improving stability.

VI. EXPERIMENTAL METHODOLOGY

For each algorithm we run the simulator with five independent random seeds. Each run lasts long enough for the election to stabilize after the fault script (typically 6–7 s of simulated time). For each run we collect:

- L_{init} : time from $t = 0.0$ s to the first election of a leader.
- L_{re} : time from the crash at $t = 2.0$ s to the first subsequent point in time when a unique leader exists after the partition heals.
- M : total count of election-related messages (ELECTION, OK, COORDINATOR, HEARTBEAT, and Raft-style votes).
- S : binary indicator of success (1 if exactly one leader exists at the end of the run, 0 otherwise).

We report median (P50) values of L_{init} and L_{re} across seeds, together with average M and the number of successful runs. The next section visualizes these outcomes.

VII. RESULTS

A. Aggregate Metrics

Table I summarizes the P50 latencies and message counts. All numbers are averaged over five seeds.

Fig. 4 visualizes the trade-off between message cost and re-election speed, while Figs. 5 and 6 provide complementary views of robustness and message overhead.

Bully achieves the lowest initial election latency (50 ms) but uses the largest number of messages: compared to Raft, it sends roughly $1168/284 \approx 4.1\times$ more messages. Multi-Attribute reduces re-election latency with respect to Bully by $(0.81 - 0.73)/0.81 \approx 9.9\%$ while cutting message count by approximately 51%. Ring’s re-election latency is nearly four times that of Multi-Attribute and it fails in 4/5 runs, as visualized in Fig. 5.

B. Latency Behavior

The latency figures can be interpreted in light of the algorithms’ designs. Bully quickly converges in a connected network because the highest-identifier node can assert dominance

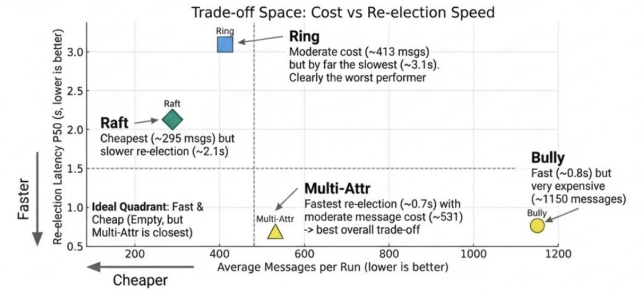


Fig. 4. Trade-off between message cost and re-election speed for the four leader-election algorithms under the crash-recovery fault script. Each marker shows the median (P50) re-election latency on the vertical axis (in seconds, lower is better) versus the average number of election-related messages per run on the horizontal axis (lower is better), aggregated over five random seeds. Raft is the cheapest (≈ 295 messages) but slower to recover (≈ 2.1 s). Bully re-elects quickly (≈ 0.8 s) but at very high cost (≈ 1150 messages). Ring has moderate cost (≈ 413 messages) but is by far the slowest (≈ 3.1 s) and also fails to converge in most runs (1/5 successes, not shown in this 2-D projection). The proposed Multi-Attribute algorithm lies closest to the ideal “fast and cheap” lower-left quadrant, combining fast re-election (≈ 0.7 s) with moderate message cost (≈ 531 messages).

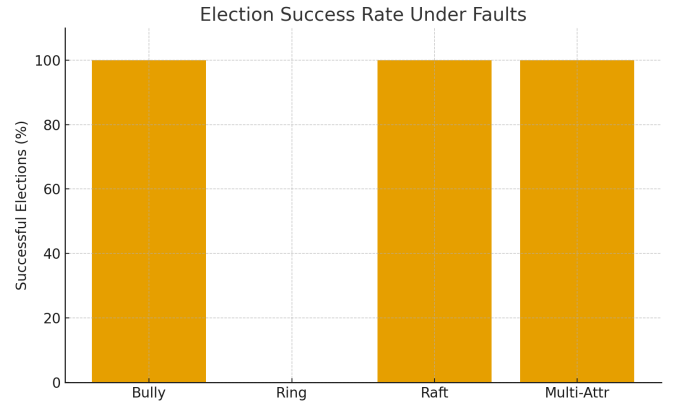


Fig. 5. Election success rate under the crash-recovery fault script. Bars show the percentage of runs (out of five seeds) that converged to a unique leader. Bully, Raft, and Multi-Attribute algorithms succeed in 100% of runs, whereas Ring fails to converge in four out of five runs.

with few round trips. However, after the crash and partition, multiple nodes simultaneously suspect failure and trigger elections. When partitions heal, another wave of elections can occur as nodes reconcile conflicting views of leadership, increasing L_{re} .

Raft’s latency is dominated by its randomized election timeouts and quorum requirement. In a partitioned state, the minority components cannot elect leaders, so some candidates repeatedly time out and restart elections. Once a majority partition forms, however, convergence is deterministic.

Multi-Attribute shares Bully’s basic structure but tends to select more stable leaders. Nodes with low battery or high CPU load are less likely to win elections, so leaders fail less frequently within our simulation window. As a result, the algorithm spends less time in repeated election phases, leading to the observed 0.73 s median re-election latency.

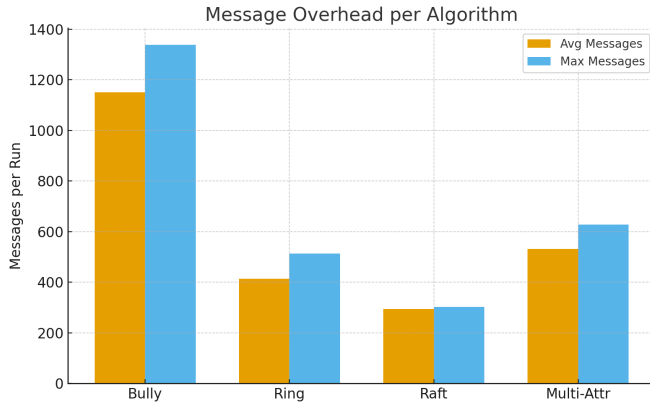


Fig. 6. Message overhead per algorithm. For each election strategy, the figure reports the average and maximum number of election-related messages observed across five runs. Bully generates the most traffic (over 1300 messages in the worst case), Raft the least (around 300), and Multi-Attribute occupies a middle ground with roughly half the overhead of Bully.

Ring’s poor performance stems from its topological assumptions. The logical ring breaks when a node or link fails; in our partitioned scenario the token can circulate only within a subset of nodes and cannot gather global information, leading to either long delays or complete failure to elect.

C. Message Overhead and Robustness

From an operator’s perspective, message overhead matters because it consumes bandwidth and CPU. Fig. 6 highlights that Raft is attractive in this dimension: at roughly 300 messages per run it uses about half the messages of Multi-Attribute and a quarter of Bully. However, Raft’s longer re-election latency (2.29 s median) may be undesirable in latency-sensitive applications.

All algorithms except Ring converge successfully in all five seeds, as shown in Fig. 5. Ring’s single successful run tends to correspond to seeds where the partition occurs at a time when the token happens to be located in the majority component; otherwise the algorithm stalls. This behavior emphasizes that algorithms assuming a static ring are fragile under even moderate failures.

VIII. DISCUSSION AND LIMITATIONS

A. Architectural Insights

The discrete-event architecture plays a crucial role in making these comparisons meaningful. By driving all algorithms through the same event scheduler, message bus, and fault injector, we ensure that differences in outcomes arise from algorithm logic rather than implementation artifacts. For example, the same sampled delay and loss pattern is replayed for each algorithm under a fixed seed, which is analogous to using the same input trace in a systems evaluation.

The architecture also clarifies where extensions could be added. For instance, dynamic timeout tuning [3] could be implemented as a feedback loop between the monitor and node modules, while knowledge-free algorithms [4] could be

evaluated simply by swapping in new node logic without modifying the bus or scheduler.

B. Threats to Validity

Our study has several limitations:

- **Scale.** With $n = 10$ nodes, asymptotic message complexity is not visible. Larger clusters could magnify the cost of Bully and reveal different trade-offs.
- **Fault diversity.** We evaluate a single crash and partition pattern. Real systems experience bursts of correlated failures, network flaps, and varying partition sizes.
- **Timeout calibration.** Timeouts are manually tuned to work reasonably with 50 ms latency. Mis-calibration could drastically change the relative performance of algorithms.
- **Simplified Multi-Attribute scoring.** Our linear combination of battery and CPU is heuristic; in practice more dimensions and non-linear functions might be required.

Despite these limitations, the experiment is internally consistent and provides a concrete, numerically grounded comparison under a well-specified model.

IX. CONCLUSION

We constructed a discrete-event simulator for leader election under crash–recovery, jitter, packet loss, and network partitions, and used it to compare Bully, Ring, Raft, and a new Multi-Attribute election algorithm. Across five seeds and a shared fault script, Multi-Attribute achieved the fastest median re-election latency (0.73 s) while using roughly half as many messages as Bully and matching Raft’s success rate. Raft remained the most message-efficient algorithm (about 300 messages per run) but required longer to recover (2.29 s). Ring was unsuitable for this environment, succeeding in only 1/5 runs.

From a design perspective, these results suggest that (i) ring-based schemes should be avoided in partition-prone environments, (ii) majority-based algorithms such as Raft provide strong robustness at moderate cost, and (iii) incorporating node capabilities into leader selection can improve stability without substantial complexity. Our simulator architecture provides a foundation for further experiments on dynamic timeouts, hierarchical election, and knowledge-free variants, extending this course project towards more complete research.

REFERENCES

- [1] “Multi-Attribute Self-Stabilizing Leader Election,” course reference from project slides.
- [2] “Hierarchical Adaptive Election in Crash-Recovery Networks,” course reference from project slides.
- [3] “Dynamic Timeout Tuning for Distributed Election,” course reference from project slides.
- [4] “Knowledge-Free Resilient Election Algorithms,” course reference from project slides.